

CS 1101 Unit 8

Programming Assignment

Assignment Response

Introduction

This assignment demonstrates how to read a dictionary from a text file, invert it so that values become keys and keys become values, and write the resulting inverted dictionary to a new output file. The program also uses try/except blocks to handle file errors gracefully, preventing the program from crashing. As *Downey (2015, p. 140)* explains in Chapter 14, *"A lot of things can go wrong when you try to read and write files,"* and it is better to handle these errors programmatically than to allow the program to terminate abruptly.

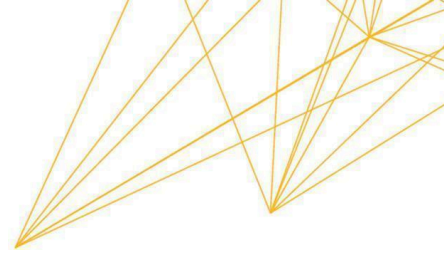
1. Input File - original_dict.txt

The input file contains the original dictionary with eight key-value pairs. Each line follows the format key: value. When multiple keys share the same value, the inversion will group them under a single key in the output file. The file is stored as a plain text file, which *Downey (2015, p. 137)* describes as *"a sequence of characters stored on a permanent medium like a hard drive."*

Contents of original_dict.txt:

apple: red

banana: yellow



cherry: red

mango: yellow

grapes: green

blueberry: blue

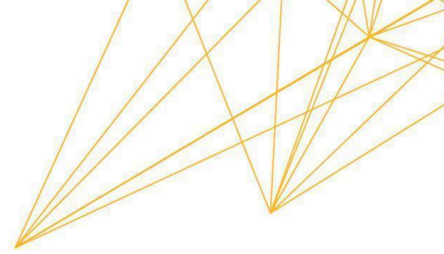
lemon: yellow

plum: purple

2. Python Program

The program below reads the original dictionary from the input file, inverts it by swapping keys and values (grouping multiple keys that share the same value), and writes the inverted dictionary to a new output file. Exception handling is implemented to catch `FileNotFoundError` if the input file does not exist, as well as a general `IOError` for any other file-related issues.

```
def read_dictionary(filepath):  
    dictionary = {}  
  
    try:  
        with open(filepath, 'r') as fin:  
            for line in fin:  
                line = line.strip()  
                if ':' in line:
```



```
        key, value = line.split(':', 1)

        dictionary[key.strip()] = value.strip()

except FileNotFoundError:

    print(f"Error: '{filepath}' was not found. Please check the filename.")

except IOError as e:

    print(f"File error: {e}")

return dictionary


def invert_dictionary(original):

    inverted = {}

    for key, value in original.items():

        if value not in inverted:

            inverted[value] = [key]

        else:

            inverted[value].append(key)

    return inverted


def write_inverted_dictionary(inverted, filepath):

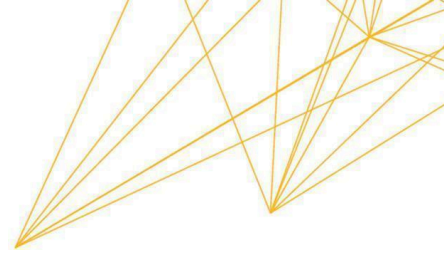
    try:

        with open(filepath, 'w') as fout:

            fout.write('\n')

            for key, values in inverted.items():

                fout.write(f"    {key}: {' '.join(values)}\n")
```



```
fout.write('{}\n')

print(f"Inverted dictionary written to '{filepath}' successfully.")

except IOError as e:

    print(f"Could not write to file: {e}")

# --- Main Program ---

input_file = 'original_dict.txt'

output_file = 'inverted_dict.txt'

print('--- Original Dictionary ---')

original = read_dictionary(input_file)

for k, v in original.items():

    print(f' {k}: {v}')

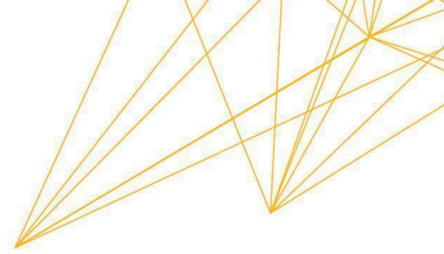
inverted = invert_dictionary(original)

print("\n--- Inverted Dictionary ---")

for k, v in inverted.items():

    print(f' {k}: {" ".join(v)}')

write_inverted_dictionary(inverted, output_file)
```



3. Output File - `inverted_dict.txt`

After running the program, the following content is written to `inverted_dict.txt`:

The screenshot shows a VS Code editor window with the following content:

```
1 {
2     red: apple, cherry
3     yellow: banana, mango, lemon
4     green: grapes
5     blue: blueberry
6     purple: plum
7 }
8
```

The terminal output shows the execution of the program:

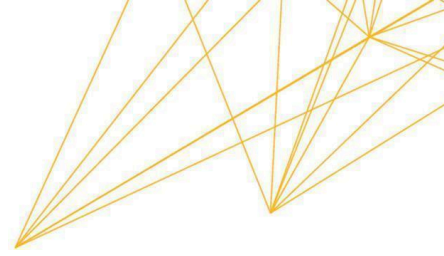
```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 8 | main ?1 | python3 dictionary_inverter.py
--- Original Dictionary ---
apple: red
banana: yellow
cherry: red
mango: yellow
grapes: green
blueberry: blue
lemon: yellow
plum: purple

--- Inverted Dictionary ---
red: apple, cherry
yellow: banana, mango, lemon
green: grapes
blue: blueberry
purple: plum
Inverted dictionary written to 'inverted_dict.txt' successfully.
```

Console Output

```
~/Doc/UoPeopleCS1101-01ProgrammingFundamentals/Programming Assignment Unit 8 | main ?1 | python3 dictionary_inverter.py
--- Original Dictionary ---
apple: red
banana: yellow
cherry: red
mango: yellow
grapes: green
blueberry: blue
lemon: yellow
plum: purple

--- Inverted Dictionary ---
red: apple, cherry
yellow: banana, mango, lemon
green: grapes
blue: blueberry
purple: plum
Inverted dictionary written to 'inverted_dict.txt' successfully.
```



4. Technical Explanation

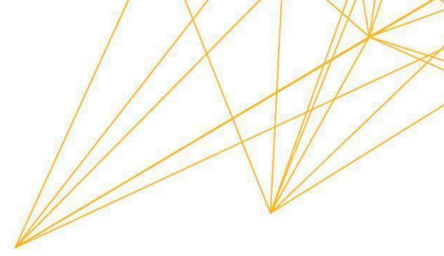
Reading the Input File

The function `read_dictionary()` opens the input file in read mode ('r') using Python's built-in `open()` function. As **Downey (2015, p.137)** explains in Chapter 14, `open()` returns a file object that provides methods for reading. According to **W3Schools (n.d.)**, the `open()` function takes two parameters - the filename and the mode, where "r" is the default read mode and "t" is the default text mode - meaning that `open('file.txt', 'rt')` and `open('file.txt')` are equivalent. Each line is stripped of leading and trailing whitespace using the `strip()` method - a necessary step because, as **Downey (2015, p. 84)** notes, file lines include a newline character ('\n') at the end. The line is then split on the colon character using `split(':', 1)`, where the second argument limits the split to one occurrence to handle values that may themselves contain colons. The resulting key and value are stripped of extra spaces and stored in the dictionary.

Exception handling is implemented using a try/except block. If the input file does not exist, Python raises a `FileNotFoundError`, which is caught explicitly and communicated to the user with a clear message. A second except clause catches `IOError`, which handles broader file-related issues such as permission errors. This approach follows the principle described by **Downey (2015, p. 140)**: *"It is better to go ahead and try - and deal with problems if they happen - which is exactly what the try statement does."*

Inverting the Dictionary

The function `invert_dictionary()` iterates over the key-value pairs of the original dictionary using the `items()` method. For each pair, it checks whether the value already exists as a key in the

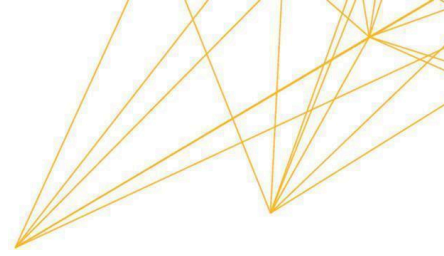


inverted dictionary. If it does not, a new list is created containing the original key as the first element. If the value already exists as a key, the original key is appended to that list. This approach correctly handles the many-to-one relationship that occurs when multiple fruits share the same color - for example, both 'apple' and 'cherry' map to 'red', and both 'banana', 'mango', and 'lemon' map to 'yellow'. The result is a dictionary where each color maps to a list of all fruits of that color.

Writing the Output File

The function `write_inverted_dictionary()` opens the output file in write mode ('w'). As **Downey (2015, p.138)** warns, opening a file in write mode clears its existing contents if the file already exists, so care must be taken when choosing the output filename. **W3Schools (n.d.)** further clarifies that the "w" mode creates the file automatically if it does not exist, which is why no separate file-creation step is needed before writing. The function writes an opening brace, then iterates over the inverted dictionary and writes each key-value pair formatted as a comma-separated list using Python's `join()` method. A closing brace is written at the end. The `write()` method, as **Downey (2015, p. 138)** notes, requires its argument to be a string, which is why the list of fruit names is joined into a single string before being written. A try/except block wraps the write operation to catch any `IOError` that might occur, such as a full disk or a permission-restricted directory. Following **Schafer (2016)**'s tutorial recommendation, the `with open(...)` context manager is used throughout, which automatically closes the file even if an error occurs during execution.

Design Decisions



Named exception types (`FileNotFoundError`, `IOError`) are used rather than a bare `except` clause.

This is intentional best practice: a bare `except` would silently catch all exceptions, including unexpected programming errors, making debugging significantly harder. By naming the exception, the handler catches only the anticipated file errors and allows all other exceptions to surface normally. This design makes the program both robust for users and transparent for developers.

623 words

References

Downey, A. (2015). *Think Python: How to think like a computer scientist* (2nd ed.). Green Tea Press. <https://greenteapress.com/thinkpython2/thinkpython2.pdf>

Schafer, C. (2016, April 29). Python tutorial: File objects - Reading and writing to files [Video]. YouTube. <https://youtu.be/Uh2ebFW8OYM>

Python File Open. (n.d.). *W3Schools*.
https://www.w3schools.com/python/python_file_handling.asp